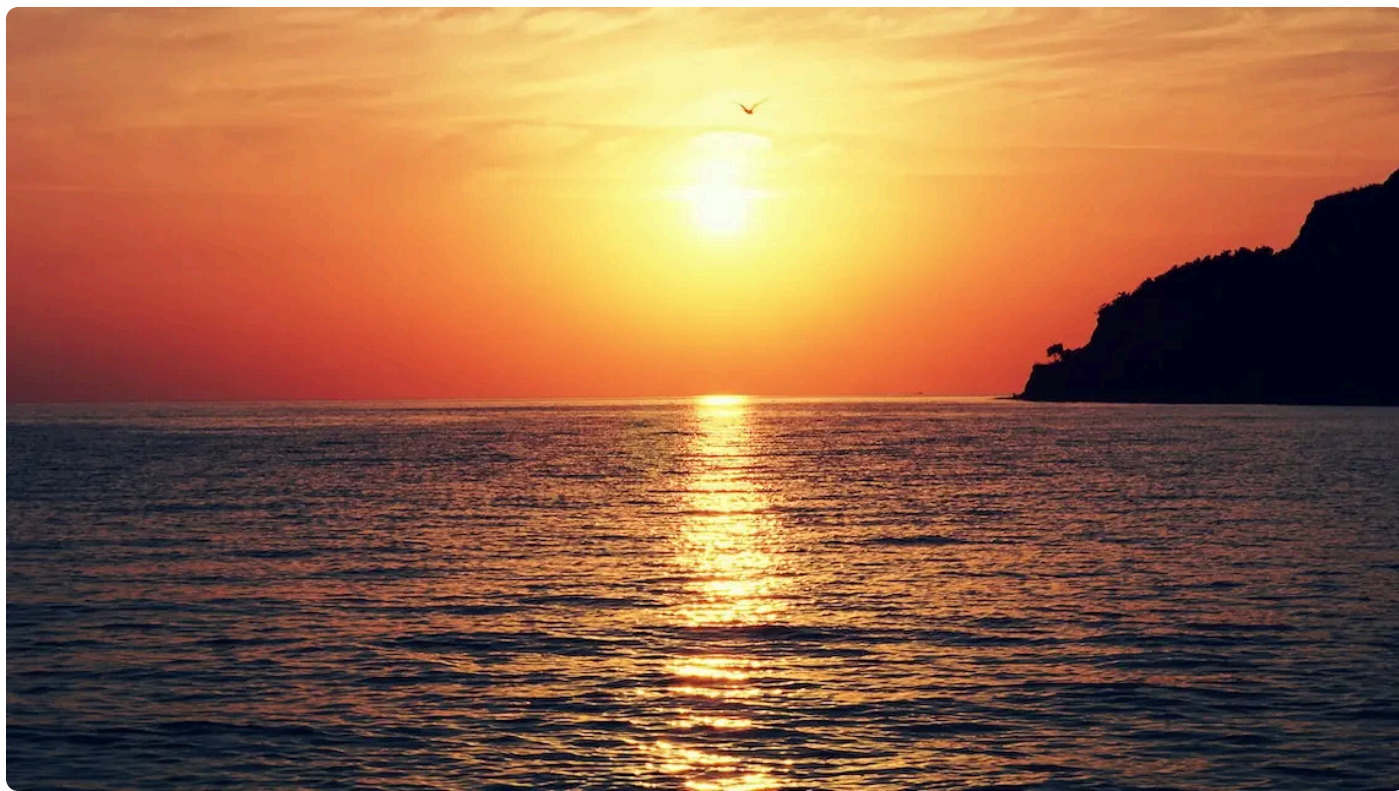


22 | 如何使用 ArgoCD 快速打造生产可用的 GitOps 工作流？

王炜 · 云原生架构与GitOps实战



你好，我是王炜。

在之前的课程中，我们学习了如何使用 Kustomize 和 Helm 来定义应用。其中，Helm Chart 在实际工作中使用较多，它有两种存储方式，一种是以源码的方式存储在 Git 仓库中，另一种是以 tgz 压缩包的方式存储在专用的 OCI 仓库中。

此外，在自动化镜像构建部分，我还向你介绍了如何使用 GitHub Action、GitLab CI 以及自托管 Tekton 来自动构建镜像。

当我们具备这些 GitOps 的核心基础后，接下来我们就可以根据实际场景，选择合适的技术栈来构建 GitOps 流水线了。

在这节课，我仍然会以示例应用为例，使用 GitHub Action 和 Helm 分别作为自动构建镜像和应用定义的工具，并通过 ArgoCD 来构建一个完整的 GitOps 工作流。

在开始今天的学习之前，你需要准备好下面这几个条件。

按照 [第 2 讲](#) 的内容在本地配置好 Kind 集群，安装 Ingress-Nginx，并暴露 80 和 443 端口。

配置好 Kubectl，使其能够访问 Kind 集群。

克隆 [kubernetes-example](#) 示例应用代码并推送到自己的 GitHub 仓库中，然后按照 [第 16 讲](#) 的内容配置好 GitHub Action 和 DockerHub Registry。

ArgoCD

在“从零上手 GitOps”这一章，我使用了 FluxCD 来构建 GitOps 流水线。FluxCD 的主要特点是比较轻量，但同时也缺少友好的 UI 控制台。相比较而言，在社区和维护方面，ArgoCD 更为活跃。所以，**在生产环境下，我推荐你使用 ArgoCD 来构建 GitOps 工作流。**

安装 ArgoCD

要使用 ArgoCD，首先需要在 Kind 集群安装它，你可以通过下面的命令来安装。

首先，创建 argocd 命名空间。

 复制代码

```
1 $ kubectl create namespace argocd
2 namespace/argocd created
```

然后，部署 ArgoCD。

 复制代码

```
1 $ kubectl apply -n argocd -f https://ghproxy.com/https://raw.githubusercontent.com
2 customresourcedefinition.apiextensions.k8s.io/applications.argoproj.io created
3 customresourcedefinition.apiextensions.k8s.io/applicationsets.argoproj.io created
4 customresourcedefinition.apiextensions.k8s.io/appprojects.argoproj.io created
5 .....
```

最后，等待 argocd 命名空间所有的工作负载处于就绪状态。

 复制代码

```
1 $ kubectl wait --for=condition=Ready pods --all -n argocd --timeout 300s
2 pod/argocd-application-controller-0 condition met
3 pod/argocd-applicationset-controller-57bfc6fdb8-x5jxc condition met
4 .....
```

请注意，由于云厂商 Kubernetes 版本存在差异，所以如果你在安装过程中发现 argocd-repo-server 工作负载一直无法启动，可以尝试删除 argocd-repo-server Deployment seccompProfile 节点的内容。

 复制代码

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: argocd-repo-server
5 spec:
6   template:
7     spec:
8       securityContext:
9         seccompProfile:
10           type: RuntimeDefault
```

安装 ArgoCD CLI

为了更加方便地配置 ArgoCD，官方还为我们提供了 CLI 工具。不同操作系统的安装方法有所差异，这里以 MacOS 和 Windows 为例简单介绍。

MacOS 可以通过 Brew 来安装。

 复制代码

```
1 $ brew install argocd
```

也可以在 [这个链接](#) 下载最新版本的 Binary 二进制文件，并移动到 `/usr/local/bin/` 目录下。

Windows 则需要在下载过后，将可执行文件移动到 PATH 下。

更详细的方法可以查看 [这份文档](#)。

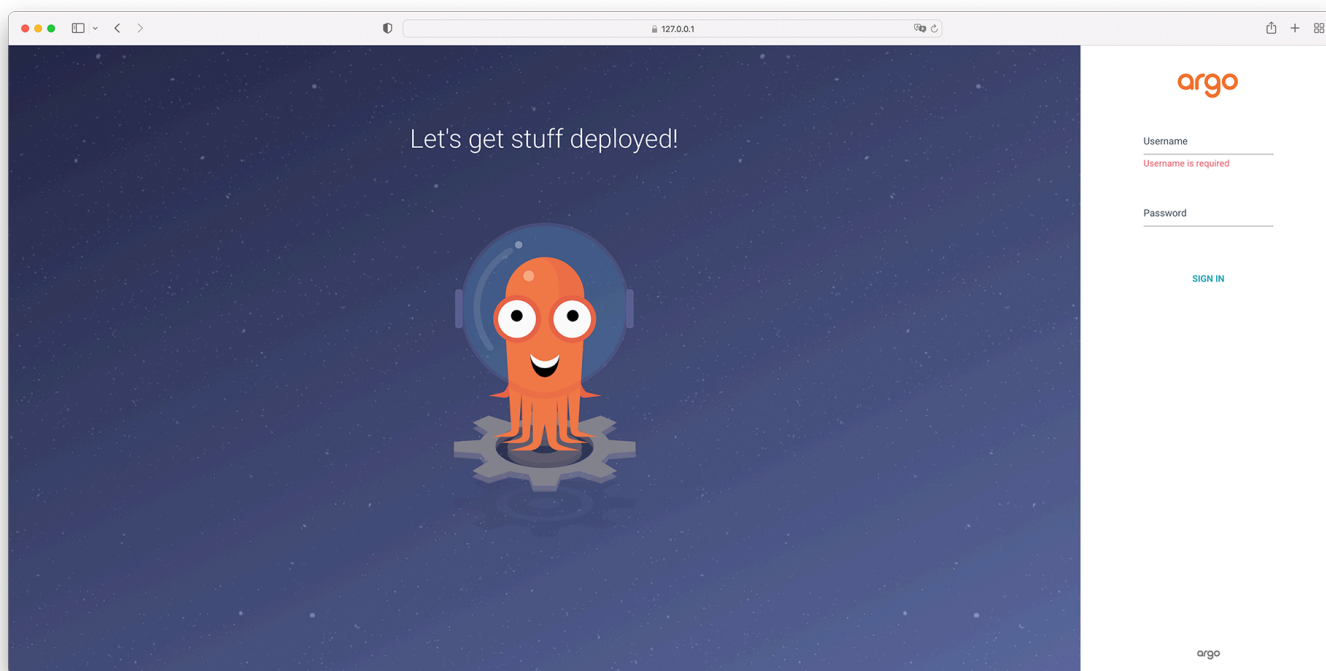
本地访问 ArgoCD

要在本地访问 ArgoCD，最简单的方式是通过端口转发来完成。你可以使用下面的命令来进行端口转发。

 复制代码

```
1 $ kubectl port-forward service/argocd-server 8080:80 -n argocd
2 Forwarding from 127.0.0.1:8080 -> 8080
3 Forwarding from [::1]:8080 -> 8080
```

接下来，使用浏览器打开：<http://127.0.0.1:8080>，这样就可以访问 ArgoCD 控制台了，如下图所示。



ArgoCD 的默认账号为 admin，密码可以通过下面的命令来获取。

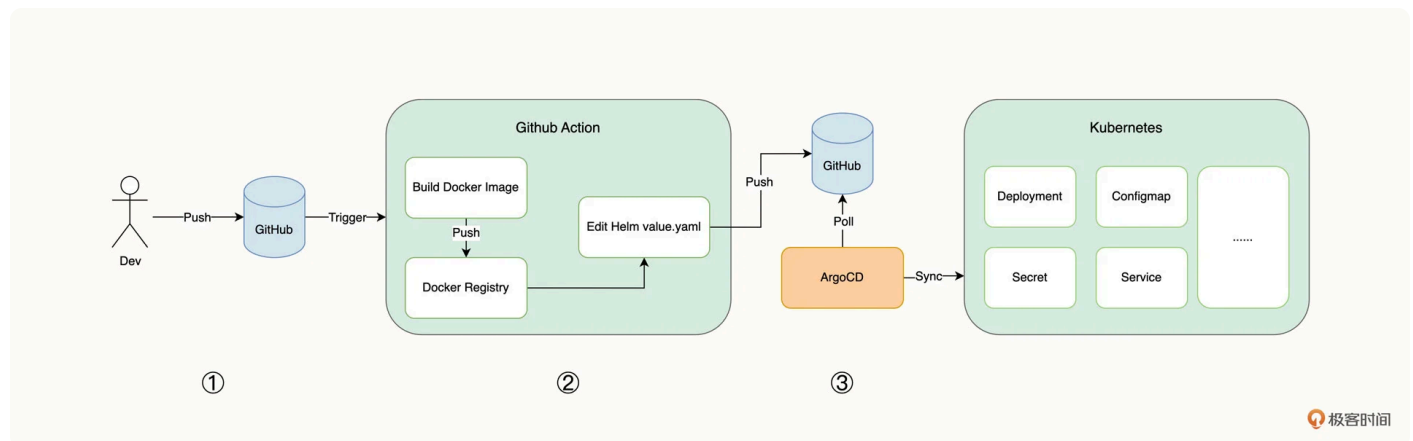
 复制代码

```
1 $ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.pa
2 Gn4b2PFG6vKm1ADm
```

登录后，即可访问 ArgoCD 的控制台。

GitOps 工作流总览

到这里，你是不是已经迫不及待想要构建工作流了？别急，在创建 GitOps 工作流之前，我们先来认识一下一个完整 GitOps 工作流都需要哪些关键步骤。



我们可以把这个完整的 GitOps 工作流分成三个部分来看。

第一部分是开发者推送代码到 GitHub 仓库，然后触发 GitHub Action 自动构建。

第二部分是 GitHub Action 自动构建，它包括下面三个步骤。

 复制代码

1. 构建示例应用的镜像。
2. 将示例应用的镜像推送到 Docker Registry 镜像仓库。
3. 更新代码仓库中 Helm Chart values.yaml 文件的镜像版本。

第三部分的核心是 ArgoCD，它包括下面两个步骤。

 复制代码

- 1 4. 通过定期 Poll 的方式持续拉取 Git 仓库，并判断是否有新的 `commit`。
- 2 5. 从 Git 仓库获取 Kubernetes 对象，与集群对象进行实时比较，自动更新集群内有差异的资源。

在之前的课程中，我们已经为示例应用创建好了 GitHub Action 来自动构建镜像，但还缺少自动更新 Helm Chart values.yaml 文件的镜像版本逻辑，我会在稍后进行配置。

现在，我们开始创建 GitOps 工作流中的第三部分，也就是创建 ArgoCD 应用，实现 Kubernetes 资源的自动同步。

创建 ArgoCD 应用

我们以示例应用为例子来创建 ArgoCD 应用，这里主要分成两个步骤。

1. 配置仓库访问权限。
2. 创建 ArgoCD 应用。

其中，如果你的示例应用仓库是公开的，可以跳过第一步。

配置 ArgoCD 仓库访问权限（可选）

在实际场景下，我们存放应用定义的仓库一般都是私有仓库，这就需要为 ArgoCD 配置仓库访问权限。

你可以通过下面的 ArgoCD CLI 工具来为 ArgoCD 添加仓库访问权限。

在使用 ArgoCD CLI 工具之前，你需要先执行 `argocd login` 命令登录。

 复制代码

- ```
1 $ argocd login 127.0.0.1:8080 --insecure
2 Username: admin
```

```
3 Password:
4 'admin:login' logged in successfully
```

注意，这里我们指定了 ArgoCD 的服务端地址为 127.0.0.1:8080，并且使用了 `--insecure` 参数来跳过 SSL 认证，你需要保持在上面运行的端口转发命令才能够顺利登录。

登录成功后，通过 `argocd repo add` 命令添加你的示例应用仓库。

 复制代码

```
1 $ argocd repo add https://github.com/lyzhang1999/kubernetes-example.git --username
2 Repository 'https://github.com/lyzhang1999/kubernetes-example.git' added
```

这里要注意将仓库地址修改为你实际的 GitHub 仓库地址，并将 `$USERNAME` 替换为 GitHub 账户 ID，将 `$PASSWORD` 替换为 GitHub Personal Token。你可以在 [这个页面](#) 创建 GitHub Personal Token，并赋予仓库相关权限，如下图所示。

## New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

### Note

What's this token for?

### Expiration \*

30 days

The token will expire on Fri, Dec 9 2022

### Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

|                                                     |                                      |
|-----------------------------------------------------|--------------------------------------|
| <input checked="" type="checkbox"/> repo            | Full control of private repositories |
| <input checked="" type="checkbox"/> repo:status     | Access commit status                 |
| <input checked="" type="checkbox"/> repo_deployment | Access deployment status             |
| <input checked="" type="checkbox"/> public_repo     | Access public repositories           |
| <input checked="" type="checkbox"/> repo:invite     | Access repository invitations        |
| <input checked="" type="checkbox"/> security_events | Read and write security events       |

## 创建 ArgoCD 应用

接下来，就可以创建 ArgoCD 应用了。ArgoCD 同时支持使用 Helm Chart、Kustomize 和 Manifest 来创建应用，这里我们以示例应用的 Helm Chart 为例。

你可以通过 `argocd app create` 命令来创建应用。

 复制代码

```
1 $ argocd app create example --sync-policy automated --repo https://github.com/lyz
2 application 'example' created
```

这里我简单解释一下每个参数的作用。

`--sync-policy` 参数代表设置自动同步策略。`automated` 的含义是自动同步，也就是说当集群内的资源和 Git 仓库 Helm Chart 定义的资源有差异时，ArgoCD 会自动执行同步操作，实时确保集群资源和 Helm Chart 的一致性。

`--repo` 参数表示 Helm Chart 的仓库地址。这里的值是示例应用的仓库地址，注意需要替换成你实际的 Git 仓库地址。

`--revision` 参数表示需要跟踪的分支或者 Tag，这里我们让 ArgoCD 跟踪 main 分支的改动。

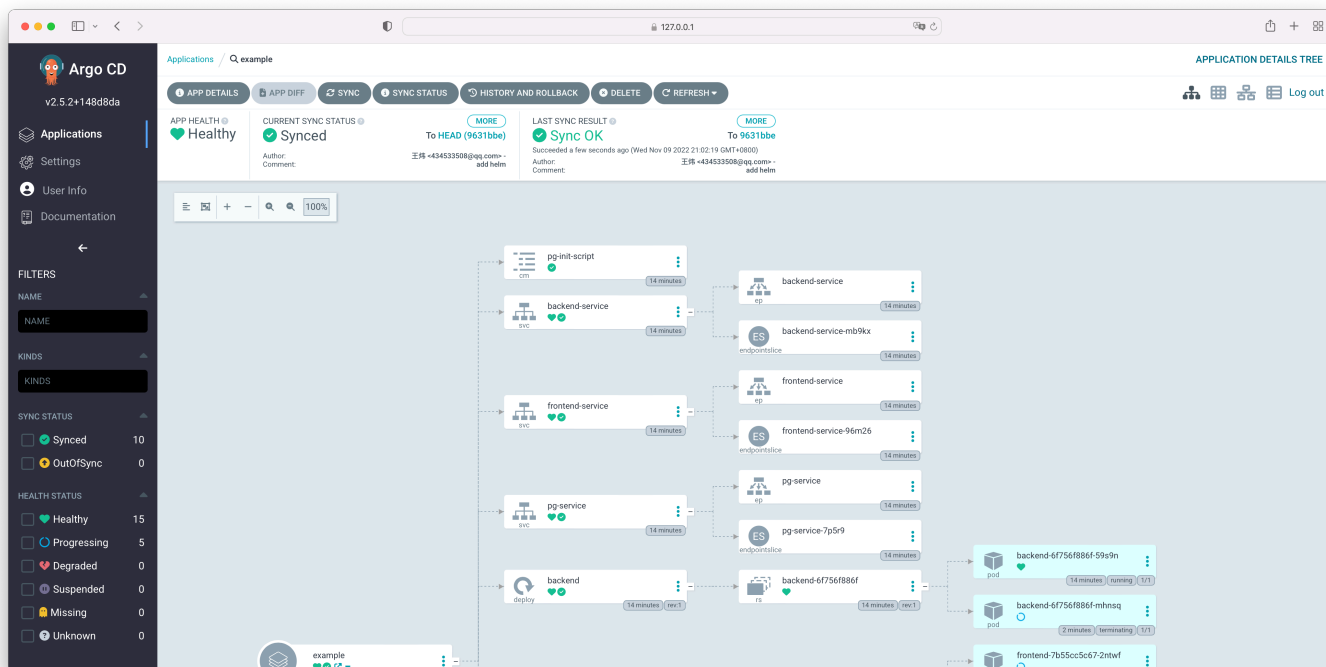
`--path` 参数表示 Helm Chart 的路径。在示例应用中，存放 Helm Chart 的目录是 `helm` 目录。

`--dest-namespace` 参数表示命名空间。这里指定了 `gitops-example` 命名空间，注意，这是一个不存在的命名空间，所以我们额外通过 `--sync-option` 参数来让 ArgoCD 自动创建这个命名空间。

最后，`--dest-server` 参数表示要部署的集群，<https://kubernetes.default.svc> 表示 ArgoCD 所在的集群。

## 查看 ArgoCD 同步状态

创建好应用之后，GitOps workflows 中的自动同步部分也就建立起来了。现在，你可以打开 ArgoCD 控制台，进入左侧的“Application”菜单来查看示例应用详情。



在应用详情页面，我们需要重点关注三个状态。

### 1. APP HEALTH: 应用整体的健康状态，它包含下面三个值。

1. Progressing: 处理中
2. Healthy: 健康状态
3. Degraded: 宕机

### 2. CURRENT SYNC STATUS: 应用定义和集群对象的差异状态，也包含下面三个值。

1. Synced: 完全同步
2. OutOfSync: 存在差异
3. Unknown: 未知

### 3. LAST SYNC RESULT: 最后一次同步到 Git 仓库的信息，包括 Commit ID 和提交者信息。

## 访问应用

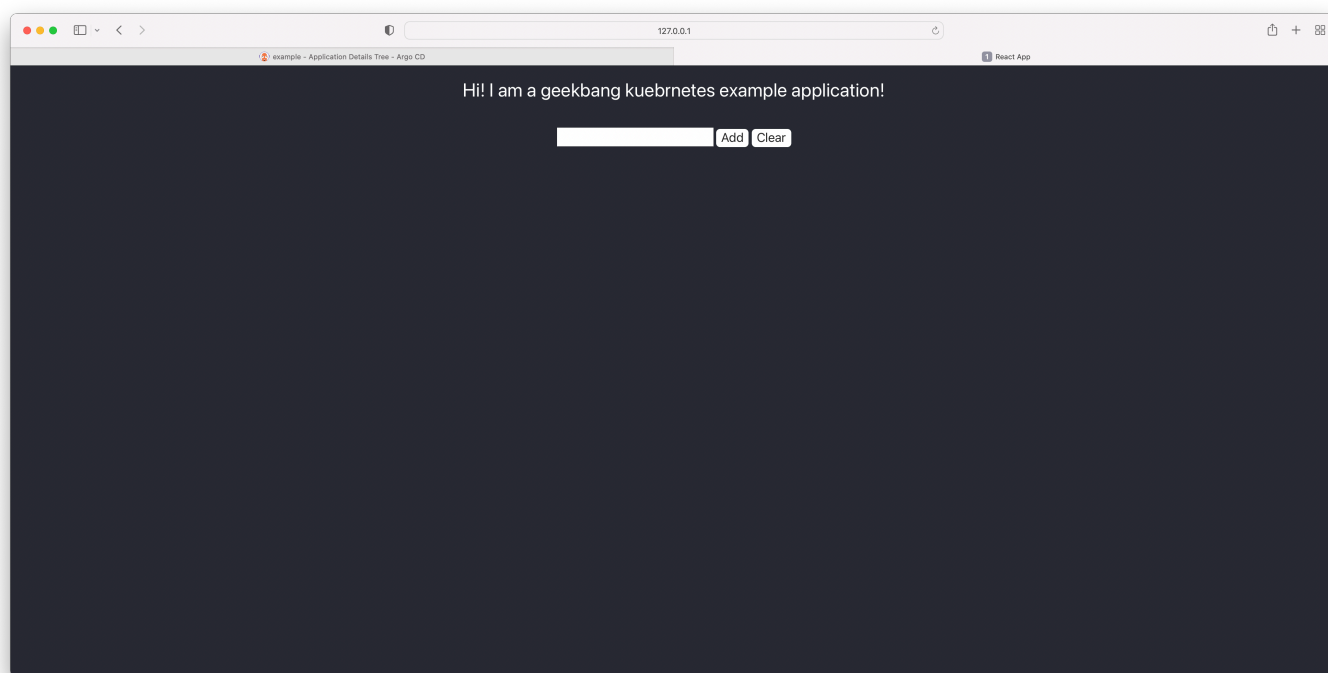
当应用健康状态变为 Healthy 之后，我们就可以访问应用了。

在这之前，如果你已经在 example 命名空间下手动部署了示例应用，为了避免 Ingress 策略冲突，你需要先删除这个命名空间。

```
1 $ kubectl delete ns example
```

 复制代码

然后，使用浏览器访问 <http://127.0.0.1>，你应该能看到示例应用的界面，如下图所示。

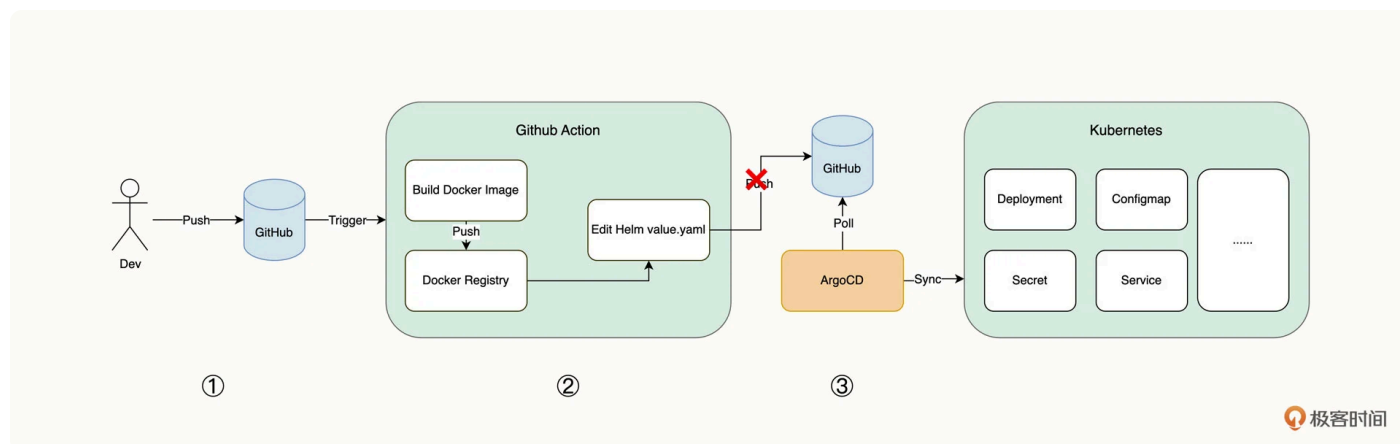


到这里，ArgoCD 部分就配置完成了。

## 连接 GitOps 工作流

在完成 ArgoCD 的应用配置之后，我们就已经将示例应用的 Helm Chart 定义和集群资源关联起来了，但整个 GitOps 工作流还缺少非常重要的一部分，就是我在上面提到的自动更新

Helm Chart values.yaml 文件镜像版本的部分，我在下面这张示意图中用“✗”把这个环节标记了出来。



在这部分 workflow 没有打通之前，提交的新代码虽然会构建出新的镜像，但是 Helm Chart 定义的镜像版本并不会产生变化，这会导致 ArgoCD 不能自动更新集群内工作负载的镜像版本。

要解决这个问题，我们还需要在 Github Action 中添加自动修改 Helm Chart 并重新推送到仓库操作。

接下来，我们修改示例应用的 .github/workflows/build.yaml 文件，在“Build frontend and push”阶段后面添加一个新的阶段，代码如下。

复制代码

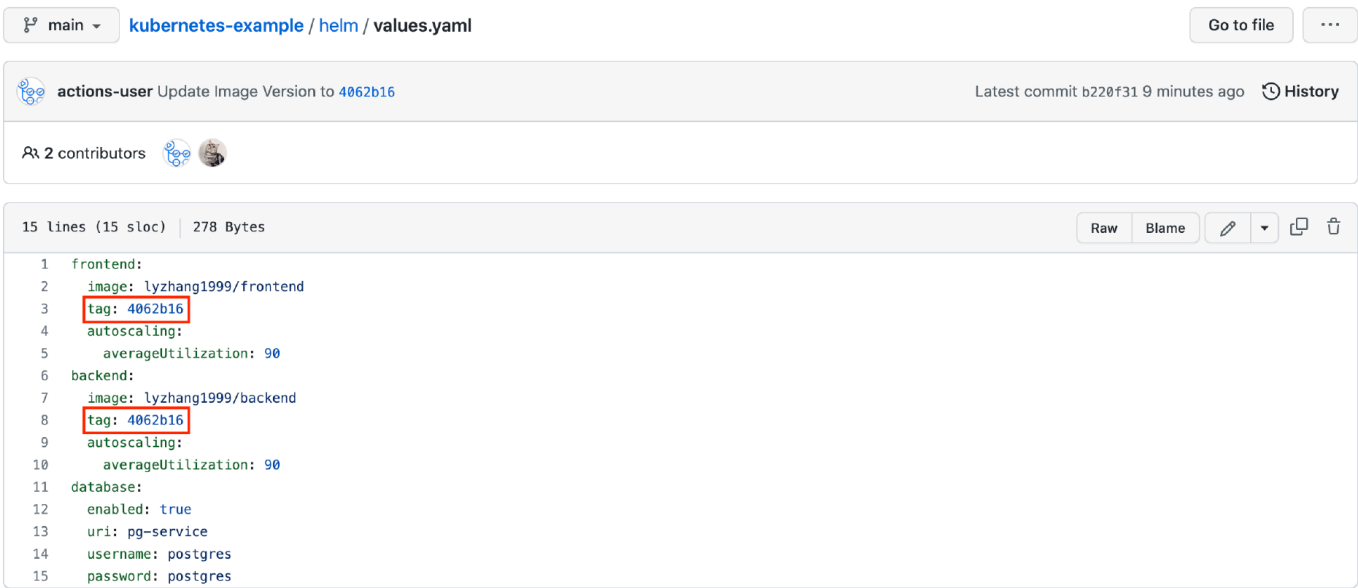
```
1 - name: Update helm values.yaml
2 uses: fjogeleit/yaml-update-action@main
3 with:
4 valueFile: 'helm/values.yaml'
5 commitChange: true
6 branch: main
7 message: 'Update Image Version to ${ steps.vars.outputs.sha_short }'
8 changes: |
9 {
10 "backend.tag": "${ steps.vars.outputs.sha_short }",
11 "frontend.tag": "${ steps.vars.outputs.sha_short }"
12 }
```

在这里，我使用了 GitHub Action 中 `yaml-update-action` 插件来修改 `values.yaml` 文件并把它推送到仓库。如果你是使用 GitLab 或者 Tekton 构建镜像，可以调用 `yq` 命令行工具来修改 YAML 文件，再使用 `git` 命令行将变更推送到仓库。

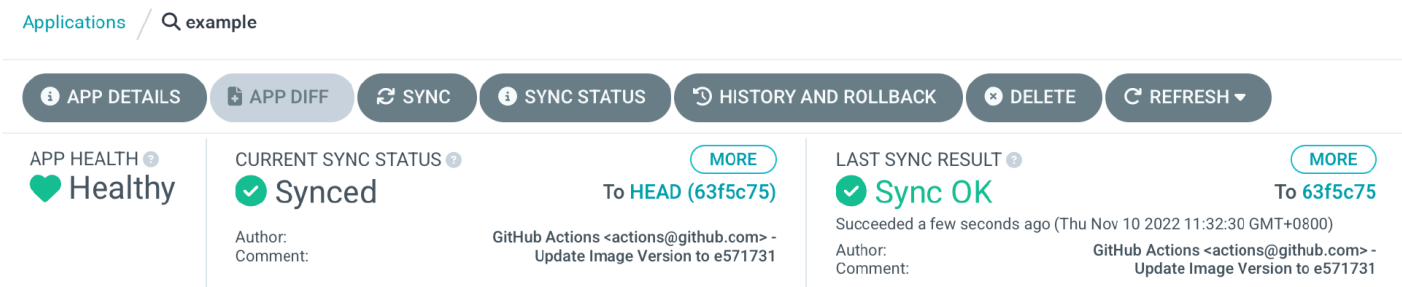
到这里，一个完整的 GitOps 工作流就建立好了。

## 体验 GitOps 工作流

接下来，你可以尝试修改 `frontend/src/App.js` 文件，例如修改文件第 49 行的“Hi! I am a geekbang”。修改完成后，**将代码推送到 GitHub 仓库 main 分支**，此时，GitHub Action 会自动构建镜像，并且还会更新代码仓库中 Helm `values.yaml` 文件的镜像版本。



ArgoCD 默认每 3 分钟会拉取仓库检查是否有新的提交，你也可以在 ArgoCD 控制台手动点击 Sync 按钮来触发同步。



ArgoCD 同步完成后，我们可以在“LAST SYNC RESULT”一栏中看到 GitHub Action 修改 values.yaml 的提交记录，当应用状态为 Healthy 时，我们就可以访问新的应用版本了。



从截图可以看出，前端界面输出内容为“Hi, I am GitOps workflow”，说明 ArgoCD 已经将新版本的应用部署到集群中了。

自此，我们体验了提交代码到构建镜像、修改应用定义和更新自动化的全流程。

## 生产建议

在生产环境下，我也给你提几个 ArgoCD 的配置建议，你可以根据实际情况来配置。

### 建议一：修改默认密码

默认情况下，ArgoCD 会在部署时生成一个随机密码，为了方便记忆和增加密码强度，你可以使用下面的命令来修改密码。

 复制代码

```
1 $ argocd account update-password
2 *** Enter password of currently logged in user (admin):
3 *** Enter new password for user admin:
```

要注意的是，在修改密码前，你需要先使用 `argocd login` 登录到 ArgoCD 服务端。

## 建议二：配置 Ingress 和 TLS

在生产环境下，为了更方便地访问 ArgoCD，你可以为它配置 Ingress，你可以将下面的 Ingress 对象部署到集群内。

 复制代码

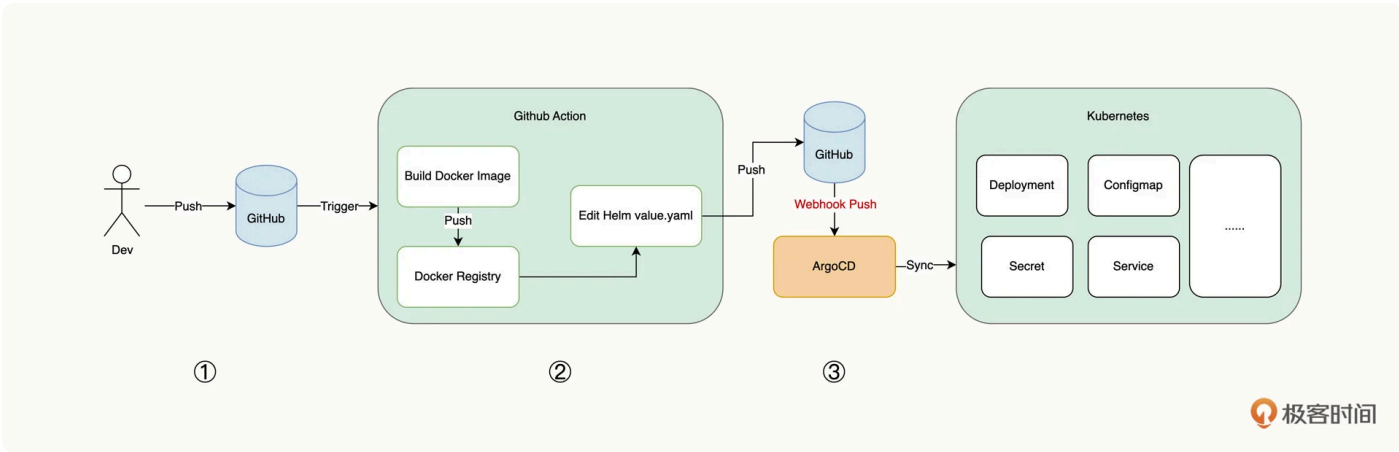
```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4 name: argocd-server-ingress
5 namespace: argocd
6 annotations:
7 cert-manager.io/cluster-issuer: letsencrypt-prod
8 kubernetes.io/ingress.class: nginx
9 kubernetes.io/tls-acme: "true"
10 nginx.ingress.kubernetes.io/ssl-passthrough: "true"
11 nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"
12 spec:
13 rules:
14 - host: argocd.example.com
15 http:
16 paths:
17 - path: /
18 pathType: Prefix
19 backend:
20 service:
21 name: argocd-server
22 port:
23 name: https
24 tls:
25 - hosts:
26 - argocd.example.com
27 secretName: argocd-secret # do not change, this is provided by Argo CD
```

此外，你还需要安装 Cert-manager，具体方法可以参考 [第 19 讲](#) 的内容。

### 建议三：使用 Webhook 触发 ArgoCD

在这节课的例子中，我们在创建应用的时候提供了参数 `--sync-policy=automated`。这时候，ArgoCD 会默认以 3 分钟一次的频率来自动拉取仓库的更改，在生产环境下，这个同步频率可能并不能满足快速发布的要求。

如果 ArgoCD 可以在公网进行访问，那么你就可以使用 ArgoCD 提供的 Webhook 触发方式来解决这个问题了，如下图所示。



和主动 Poll 模型不同的是，源码仓库在收到开发者推送代码的事件后，将实时通过 HTTP 请求来通知 ArgoCD，也就是图中红色字体的部分。

要使用 Webhook 通知的方式，首先你需要在源码仓库进行配置。以 GitHub 为例，首先进入仓库的“Settings”页面，点击左侧的“Webhook”菜单进入配置页面，如下图所示。

## Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

### Payload URL \*

### Content type

### Secret

### SSL verification

 By default, we verify SSL certificates when delivering payloads.

☒ **Enable SSL verification** ☐ **Disable (not recommended)**

### Which events would you like to trigger this webhook?

- ☒ Just the push event.
- ☐ Send me **everything**.
- ☐ Let me select individual events.

### ☒ Active

We will deliver event details when this hook is triggered.

Add webhook

在 Payload URL 中输入你的 ArgoCD Server 外网访问域名，/api/webhook ArgoCD 专门用于接收外部 Webhook 消息的固定路径。

Content type 选择 application/json，并在 Secret 中输入你要配置的 Webhook 的密钥，这个密钥需要提供给 ArgoCD 来校验 Webhook 来源是否合法。

接下来点击“Add webhook”就可以保存了。

接下来，你还需要为 ArgoCD 提供 GitHub Webhook 密钥，使用下面的命令来编辑 argocd-secret 对象。

 复制代码

```
1 $ kubectl edit secret argocd-secret -n argocd
```

并将 GitHub Webhook 的密钥加入到 Secret 对象的内容里。

 复制代码

```
1 apiVersion: v1
2 kind: Secret
3 metadata:
4 name: argocd-secret
5 namespace: argocd
6 type: Opaque
7 data:
8 ...
9 stringData:
10 # 加入这一项
11 webhook.github.secret: my-secret
```

注意，stringData 可以直接输入 Webhook Secret 内容而不需要进行 Base64 编码。

如果你使用的是其他的代码托管平台，例如 GitLab，可以参考 [这份文档](#) 进行配置。

## 建议四：将源码仓库和应用定义仓库分离

为了方便演示，我将示例应用的源码和 Helm Chart 存储在了同一个 Git 仓库，实际上，**这并不是一个好的实践**。

这种方案有两个比较大的问题。首先，当我们手动修改 Helm Chart 并推送到 Git 仓库之后，在业务代码不变的情况下也会触发应用镜像构建，这个过程是没有必要的。

其次，在有一定规模的团队中，开发和发布过程是分开的，应用定义仓库一般只有基础架构部门或者 SRE 部门具有修改权限，将源码和应用定义放在同一个 Git 仓库不利于权限控制，开发者也很容易误操作。

所以，基于上面这两个问题，我强烈建议你**将业务代码和应用定义分开存储**。

## 建议五：加密 GitOps 中存储的密钥

在示例应用中，我使用的是 DockerHub 公开仓库，所以 Kubernetes 集群不需要镜像拉取凭据就可以拉取到镜像。

在实际生产环境下，一般我们会使用内部自建例如 Harbor 私有仓库。所以在大部分情况下，我们会在 Helm Chart 里增加一个包含镜像拉取凭据的 Secret 对象。

 复制代码

```
1 apiVersion: v1
2 kind: Secret
3 metadata:
4 name: regcred
5 type: kubernetes.io/dockerconfigjson
6 data:
7 .dockerconfigjson: >-
8 eyJhdXRocyI6eyJodHRwczovL2luZGV4LmRvY2tldCI5pby92MS8i0nsidXNlcm5hbWUiOiJseXpoY
```

这样，当 ArgoCD 部署应用时，会一并将拉取凭据部署到集群中，这就解决了镜像拉取权限的问题。

但是，Secret 对象并没有加密功能，**这可能会导致凭据泄露**。所以，我们需要对这些敏感信息进行加密处理。关于如何加密密钥，我会在后续“多环境管理和安全”章节为你详细介绍。

## 总结

在这节课，我以示例应用为例，将 GitHub Action、应用定义以及 ArgoCD 连接起来，构建了完整的 GitOps 工作流。

通过这个例子我们会发现，建立完整的 GitOps 工作流涉及到下面这几个技术：

1. Docker 镜像
2. CI 构建
3. 镜像仓库
4. 应用定义
5. Kubernetes
6. ArgoCD

在建立 GitOps 工作流的过程中，通常有两个难点。**第一是如何进行技术选型和组合，第二是如何将它们连接起来。**

我先总结一下如何解决第一个问题：技术选型和组合。

在这节课的例子中，我使用了 GitHub Action 作为 CI 构建工具，此外，你还可以选择 GitLab CI 或者自托管的 Tekton 。对于镜像仓库，我使用了 DockerHub 来存储镜像，你也可以使用 GitHub Package 或者自建 Harbor 来存储镜像。最后，在应用定义方面，我使用了 Helm Chart 作为例子，你还可以选择 Kustomize 甚至是 Kubernetes Manifest 。

在建立 GitOps 的过程中，你可以根据团队实际的情况，对这些技术栈任意组合。

对于如何连接的问题，它最底层的追问是，在构建好镜像并将其推送到镜像仓库之后，怎么通知 ArgoCD 部署新镜像版本。

在这节课的例子中，我在 GitHub Action 里修改了 values.yaml 文件，并将其推送到了 Git 仓库，所以会产生新的 commit，进而达到通知 ArgoCD 的效果。

**这种方式虽然能解决问题，但在一些场景下可能并不适合。**例如，在开发和发布相对独立的场景下，因为权限和安全的问题可能并不允许 CI 直接修改应用定义。在这种情况下，我们就可以使用另外一种方式来通知 ArgoCD，也就是 **Argo CD Image Updater**。它可以通过监听镜像版本的更新来触发 ArgoCD 同步，我们会在下一节课详细介绍。

## 思考题

最后，给你留一道思考题吧。

在 ArgoCD 自动同步完成后，应用的“CURRENT SYNC STATUS”会从“Synced”很快变为“OutOfSync”，这是为什么呢？如何解决？

欢迎你给我留言交流讨论，你也可以把这节课分享给更多的朋友一起阅读。我们下节课见。

### AI智能总结

本文以示例应用为例，介绍了如何使用 ArgoCD 构建完整的 GitOps 工作流。通过结合 GitHub Action、应用定义以及 ArgoCD，读者可以快速了解如何建立 GitOps 工作流，涉及到 Docker 镜像、CI 构建、镜像仓库、应用定义、Kubernetes 和 ArgoCD 等技术。在技术选型和组合方面，读者可以根据团队实际情况任意组合技术栈。在连接技术方面，文章介绍了如何通知 ArgoCD 部署新镜像版本，以及介绍了 Argo CD Image Updater 的使用方式。最后，文章提出了一个思考题，邀请读者交流讨论。整篇文章以实际操作为主，适合读者快速了解和实践，同时提供了一些生产建议，有助于读者更好地使用 ArgoCD 构建生产级的 GitOps 工作流。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

## 全部留言 (18)

最新 精选



邵涵

2023-03-20 来自北京

对于思考题，CURRENT SYNC STATUS 从 Synced 变为 OutOfSync 可能是应用定义中的 Deployment 的 replicas 指定的数量与 k8s 集群中该 Deployment 下 pod 的实际数量不符造成的吗？因为集群中 Deployment 下的 pod 实际数量可能是 HPA 动态管控的。如果是这个原因的话，那是有什么方法可以声明忽略对这个数量的比对吗？

另外，还有两个问题，使用 ArgoCD 做自动的拉取、比对，和更新 k8s 集群中对象

1. 如何指定给某个环境使用的 values-xxx.yaml 文件？
2. 如果是要同时更新多个环境，是要为每个环境都创建一个 ArgoCD 应用吗？

作者回复: 回答正确。解决方案也非常简单，把 Deployment 的 replicas 字段删除即可，这样该字段会由 HPA 的最小副本数控制。

另外两个问题提的非常好，你可以参考 27 讲的内容，多环境自动创建的方法。



👍 2



**gyl1989113**

2023-02-13 来自四川

argocd repo add https://github.com/xxx/kubernetes-example.git --username xxx --password xxx 为什么我执行这句报错rpc error: code = Unknown desc = error testing repository connectivity: authentication required

作者回复: 尝试用 git 协议，这个仓库是公开的。

共 2 条评论 >

👍 1



**无名无姓**

2023-01-27 来自北京

这个流程可以放在一个平台里面监控么

作者回复: 没有开源方案，可以自研~



👍 1



**Sophia-百鑫**

2023-08-02 来自上海

老师好，如果技术栈是 gitlabCI pipeline +ArgoCD,应用定义 helm chart，  
如果修改 gitlab-ci.yaml文件，使CI 自动修改 helm values.yaml 文件的 image tag ?

作者回复: 是一种方案，另一种方案是使用 argo image updater 自动监听镜像版本的变化。



👍



**辣条愛乃**

2023-04-07 来自湖北

请教下，"连接GitOps工作流"这一步，我使用的gitlab，但是似乎没有找到，类似github的"valueFile"用来修改helm里面的tag id，不知道gitlab是怎么修改的

作者回复: 把 gitlab ci 当做是一台机器, 本质上可以通过 shell 命令实现任何操作, 比如编辑 yaml 文件可以用 yq 命令行工具。



辣条愛乃

2023-03-28 来自湖北

自己搭建的gitlab, 但是报错

```
FATA[0003] rpc error: code = Unknown desc = error testing repository connectivity: Get "https://www.gittest.com/root/kubernetes-example.git/info/refs?service=git-upload-pack": dial tcp 128.14.151.194:443: connect: connection refused
```

作者回复: 看着是网络原因导致的, 你可以尝试用托管的 GitLab 跑流程。



大圈

2023-03-20 来自北京

感谢老师的解答, 还有一个问题请教一下:

我们目前直接使用jenkins做CI/CD, 然后我准备把CD工具替换为argocd。

关于"自动更新 Helm Chart values.yaml 文件镜像版本的部分"这块实践, 看老师的教程可得github、gitlab-ci都可以实现。但是我暂时不想用gitlab-ci做CI工具, 继续使用Jenkins做CI操作的话能否实现"自动更新 Helm Chart values.yaml 文件镜像版本的部分"呢?

我目前想到的方案就是通过自己写脚本来实现了。

作者回复: 很容易实现, Jenkins 里通过 shell 命令调用 YQ 工具来更新 helm chart, 核心原理不变, 只要能更新 values.yaml 即可。



大圈

2023-03-15 来自北京

请教一下, 假如我有3个java服务, 每个服务对应3套环境, test,preview,product, 那么我是不是就得给这3个服务分别创建3个application, 在argocd上。

有什么好的方式来解决在argocd上重复创建application的问题吗?

作者回复: 非常好的问题, 你可以查看第27讲, 这里面有提到多环境管理。



**bob**

2023-03-02 来自广东

老师, 您好! 请问后端运行报错会是什么原因? 谢谢!

Traceback (most recent call last):

File "/usr/local/lib/python3.8/runpy.py", line 194, in \_run\_module\_as\_main

return \_run\_code(code, main\_globals, None,

File "/usr/local/lib/python3.8/runpy.py", line 87, in \_run\_code

exec(code, run\_globals)

File "/usr/local/lib/python3.8/site-packages/flask/\_\_main\_\_.py", line 3, in <module>  
main()

File "/usr/local/lib/python3.8/site-packages/flask/cli.py", line 1050, in main  
cli.main()

File "/usr/local/lib/python3.8/site-packages/click/core.py", line 1055, in main  
rv = self.invoke(ctx)

File "/usr/local/lib/python3.8/site-packages/click/core.py", line 1657, in invoke  
return \_process\_result(sub\_ctx.command.invoke(sub\_ctx))

File "/usr/local/lib/python3.8/site-packages/click/core.py", line 1404, in invoke  
return ctx.invoke(self.callback, \*\*ctx.params)

File "/usr/local/lib/python3.8/site-packages/click/core.py", line 760, in invoke  
return \_\_callback(\*args, \*\*kwargs)

File "/usr/local/lib/python3.8/site-packages/click/decorators.py", line 84, in new\_func  
return ctx.invoke(f, obj, \*args, \*\*kwargs)

File "/usr/local/lib/python3.8/site-packages/click/core.py", line 760, in invoke  
return \_\_callback(\*args, \*\*kwargs)

File "/usr/local/lib/python3.8/site-packages/flask/cli.py", line 911, in run\_command  
raise e from None

File "/usr/local/lib/python3.8/site-packages/flask/cli.py", line 897, in run\_command  
app = info.load\_app()

File "/usr/local/lib/python3.8/site-packages/flask/cli.py", line 312, in load\_app  
app = locate\_app(import\_name, None, raise\_if\_not\_found=False)

File "/usr/local/lib/python3.8/site-packages/flask/cli.py", line 218, in locate\_app  
\_\_import\_\_(module\_name)

File "/app/app.py", line 18, in <module>

```
db.init_app(app)
File "/usr/local/lib/python3.8/site-packages/flask_sqlalchemy/extension.py", line 253,
in init_app
 raise RuntimeError(
RuntimeError: A 'SQLAlchemy' instance has already been registered on this Flask app. I
mport and use that instance instead.
```

作者回复: 示例应用的后端服务的镜像版本 latest 和 1276e23 都是运行正常的, 请问你使用的镜像版本是?

共 5 条评论 >



**加菲老猫**

2023-02-22 来自上海

老师您好, 我尝试根据您的范例, 在github action中 Update helm values.yaml遇到以下错误, 请您给个思路, 谢谢!

```
updateFile is deprecated, the updated content will be written to the file by default from now on
```

```
Error: HttpError: Resource not accessible by integration
```

作者回复: 把 updateFile 参数删除即可。

